

Phase 1: Custom Networking Foundation (VPC)

We will build a dedicated, isolated virtual network to host our infrastructure instead of using the default configuration.

1. Create the Custom VPC

1. Open the **AWS Console**, search for **VPC**, and click on **VPCs** under the Virtual Private Cloud menu.
2. Click the orange **Create VPC** button in the top right.
3. Under *Resources to create*, select **VPC only**.
4. **Name tag:** Type `soar-production-vpc`.
5. **IPv4 CIDR block:** Select *IPv4 CIDR manual input* and type `10.0.0.0/16`.
6. Leave everything else as default and click **Create VPC** at the bottom.

2. Provision Two Public Subnets

An Application Load Balancer requires at least two subnets in different physical datacenters (Availability Zones) to route traffic.

1. In the left-hand menu of the VPC console, click **Subnets**, then click **Create subnet** in the top right.
2. **VPC ID:** Click the dropdown and select `soar-production-vpc`.
3. **Subnet 1 Settings:**
 - **Subnet name:** `soar-public-subnet-1a`
 - **Availability Zone:** Select your region's first zone (e.g., `us-east-1a`).
 - **IPv4 CIDR block:** Type `10.0.1.0/24`.
4. Click **Add new subnet** at the bottom of the page to add the second one:
 - **Subnet name:** `soar-public-subnet-1b`
 - **Availability Zone:** Select your region's second zone (e.g., `us-east-1b`).
 - **IPv4 CIDR block:** Type `10.0.2.0/24`.
5. Click **Create subnet** at the bottom.

3. Deploy the Internet Gateway

To allow users to access your web application, your custom VPC needs a gateway to the public internet.

1. In the left-hand menu, click **Internet gateways**, then click **Create internet gateway**.
2. **Name tag:** Type soar-vpc-igw.
3. Click **Create internet gateway**.
4. On the next screen showing your new gateway, click the **Actions** dropdown menu (top right) select **Attach to VPC**.
5. **Available VPCs:** Select soar-production-vpc and click **Attach internet gateway**.

4. Configure Routing Controls

1. In the left-hand menu, click **Route tables**.
2. Find the route table associated with your soar-production-vpc (you can check the VPC ID column). Click its checkbox, click the **Actions** dropdown select **Edit routes**.
3. Click **Add route**:
 - **Destination:** Type 0.0.0.0/0.
 - **Target:** Select **Internet Gateway**, then click on your soar-vpc-igw.
4. Click **Save changes**.

Phase 2: Structural Access Security Boundaries

We will build three independent security groups. This multi-tier setup prevents anyone from hitting your server directly, forcing them to go through the ALB and WAF proxy layer first.

Go to **VPC Console Security groups Create security group**.

Group 1: soar_production_alb_sg

- **Security group name:** soar_production_alb_sg
- **Description:** Public HTTP edge access for the Load Balancer
- **VPC:** Delete the default value, and select your custom soar-production-vpc.
- **Inbound rules:** Click *Add rule*:

- **Type:** HTTP | **Port:** 80 | **Source:** Anywhere-IPv4 (0.0.0.0/0)
- Scroll to the bottom and click **Create security group**.

Group 2: soar_production_target_sg

- **Security group name:** soar_production_target_sg
- **Description:** Production backend server rule set
- **VPC:** Select your custom soar-production-vpc.
- **Inbound rules:** Click *Add rule* twice to configure two distinct paths:
 1. **Rule 1 (Web):** Type: HTTP | Port: 80 | Source: Select **Custom**, click inside the search box, and select **soar_production_alb_sg**. *(This guarantees that your backend only responds to your proxy firewall).*
 2. **Rule 2 (Admin Management):** Type: SSH | Port: 22 | Source: Select **My IP**. *(Locks down administration capabilities to your machine).*
- Scroll to the bottom and click **Create security group**.

Group 3: soar_production_quarantine_sg

- **Security group name:** soar_production_quarantine_sg
- **Description:** Containment cell with zero ingress paths
- **VPC:** Select your custom soar-production-vpc.
- **Inbound rules:** **Do not add any rules.** Leave this completely empty.
- Scroll to the bottom and click **Create security group**.

Phase 3: Web Server & Target Application Setup

1. Launch the EC2 Instance

1. Search for **EC2** in the top AWS search bar and click on it. Click **Instances** in the left-hand menu, then click the orange **Launch instances** button.
2. **Name:** Type soar_production_web_server.
3. **Application and OS Images:** Select the default **Ubuntu** (Ubuntu Server 24.04 LTS).
4. **Instance type:** Select t2.micro (Free tier eligible).

5. **Key pair:** Select an existing key pair or generate a new one so you can SSH in.
6. **Network settings:** Click the **Edit** button on the right:
 - **VPC:** Change this to your custom soar-production-vpc.
 - **Subnet:** Choose soar-public-subnet-1a.
 - **Auto-assign public IP:** Change this dropdown to **Enable**.
 - **Firewall (security groups):** Choose **Select existing security group**, then check the box next to **soar_production_target_sg**.
7. Click **Launch instance** in the summary panel on the right. Copy its **Public IP address** from your instance list once active.

2. DVWA inside Docker

Open your local computer's terminal or command prompt and SSH directly into your new server:

```
ssh -i "your-key.pem" ubuntu@YOUR_EC2_PUBLIC_IP
```

Once connected, execute these commands to download and spin up the vulnerable web target:

```
# Refresh system indices and install the container architecture
```

```
sudo apt update -y && sudo apt install docker.io -y
```

```
sudo systemctl start docker && sudo systemctl enable docker
```

```
# Launch Damn Vulnerable Web Application bound to host port 80
```

```
sudo docker run --rm -d -p 80:80 vulnerables/web-dvwa
```

Phase 4: High-Availability Proxy Topology

We need to build the Target Group and Application Load Balancer to act as our front-line gatekeeper.

1. Provision the Target Group

1. Scroll down the left-hand EC2 menu, locate **Load Balancing**, and click **Target Groups**. Click **Create target group**.
2. **Choose a target type:** Select **Instances**.
3. **Target group name:** Type `soar-production-tg`.
4. **Protocol / Port:** Keep as HTTP / Port 80.
5. **VPC:** Ensure you switch this dropdown to your custom **soar-production-vpc**. Click **Next**.
6. Under *Available instances*, click the checkbox next to `soar_production_web_server`, click **Include as pending below**, and click the blue **Create target group** button.

2. Stand Up the Application Load Balancer (ALB)

1. In the left-hand EC2 menu, click **Load Balancers**, then click **Create load balancer** select **Application Load Balancer**.
2. **Load balancer name:** Type `soar-production-alb`.
3. **Scheme:** Keep as *Internet-facing*.
4. **Network mapping (VPC):** Select your custom `soar-production-vpc`.
5. **Mappings (Subnets):** Check the boxes for **both** Availability Zones (`us-east-1a` and `us-east-1b`) and map them to their corresponding subnets (`soar-public-subnet-1a` and `soar-public-subnet-1b`).
6. **Security groups:** Click the small "X" to remove the default group, and select **soar_production_alb_sg**.
7. **Listeners and routing:** Under *Default action*, select the dropdown and choose your **soar-production-tg**.
8. Click **Create load balancer** at the bottom. Copy the **DNS name** string generated for your ALB (e.g., `soar-production-alb-123.amazonaws.com`). Paste it into a new browser tab to verify you can reach the DVWA homepage!

Phase 5: Web Application Firewall & Forensic Storage

1. Build the S3 Logging Vault

1. Search for **S3** in the top AWS search bar, click it, and hit **Create bucket**.
2. **Bucket name:** Type exactly **aws-waf-logs-soar-production**. *(If AWS flags this as taken globally, simply append a unique number to the end, like aws-waf-logs-soar-production-99).*
3. Leave all options at default and click **Create bucket**.

2. Deploy AWS WAF (Web Application Firewall)

1. Search for **WAF & Shield** in the top AWS search bar, click it, and go to **Web ACLs** in the left menu. Click **Create web ACL**.
2. **Resource type:** Regional resources. Select your region from the dropdown (e.g., US East (N. Virginia)).
3. **Name:** Type soar-production-waf.
4. **Associated AWS resources:** Click *Add AWS resources*, select **Application Load Balancer**, check your soar-production-alb, click Add. Click Next.
5. **Add rules and rule groups:** Click the *Add rules* dropdown button, choose **Add managed rule groups**:
 - Scroll down to *Free rule groups*, find **Core rule set** and click **Add to web ACL**.
 - Find **Known bad inputs** and click **Add to web ACL**.
 - Click the blue **Add rules** button at the bottom of the page.
6. Verify the *Default web ACL action* at the bottom is checked as **Allow**. Click Next through the screens until you reach Step 4.

3. Bind WAF Logging to Your S3 Bucket

1. On **Step 4: Configure metrics**, scroll down to the **Logging** subsection.
2. Click the **Enable logging** toggle button.
3. **Logging destination:** Select **Amazon S3 bucket**.
4. **S3 bucket:** Paste your bucket's ARN path (e.g., arn:aws:s3:::aws-waf-logs-soar-production).

5. Click Next and then hit **Create web ACL**.

Phase 6: The Automated SOAR Automation Engine

1. Establish the Lambda IAM Privilege Role

1. Search for **IAM** in the top search bar, click it, go to **Policies** in the left menu, and click **Create policy**.
2. Click the **JSON** tab in the top right of the editor box. Delete everything inside it and paste this exact policy block:

JSON

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEC2NetworkContainment",
      "Effect": "Allow",
      "Action": [
        "ec2:ModifyInstanceAttribute",
        "ec2:DescribeInstances"
      ],
      "Resource": "*"
    },
    {
      "Sid": "AllowReadingWafLogs",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::aws-waf-logs-soar-production",
        "arn:aws:s3:::aws-waf-logs-soar-production/*"
      ]
    }
  ]
}
```

3. Click **Next**, name the policy `soar_production_lambda_policy`, and click **Create policy**.
4. Click **Roles** in the left IAM menu, then click **Create role**.
5. Select **AWS Service** select **Lambda** from the service dropdown. Click **Next**.
6. In the search box, find and check the box next to **soar_production_lambda_policy**. Also search for and check **AWSLambdaBasicExecutionRole**. Click **Next**.
7. Name the role `soar_lambda_execution_role` and click **Create role**.

2. Deploy the Lambda Engine Code

1. Search for **Lambda** in the top search bar, click it, and hit **Create function**.
2. **Function name:** Type `soar_production_containment_engine`.
3. **Runtime:** Select **Python 3.12** (or your latest stable Python environment version).
4. **Permissions:** Expand *Change default execution role* select **Use an existing role** choose your **soar_lambda_execution_role**. Click **Create function**.
5. In the Code window under the *Code* tab, double-click `lambda_function.py`, clear the default code placeholder text completely, and paste this robust log-looping, base64-compliant production automation engine:

Python

```
import json
import boto3
import urllib.parse
import urllib.request
import gzip
import io
import base64

s3_client = boto3.client('s3')
ec2_client = boto3.client('ec2')
ec2_resource = boto3.resource('ec2')

def lambda_handler(event, context):
    print("Received Live WAF Event: " + json.dumps(event))

    # ===== CONFIGURATION BLOCK =====
    # Paste your exact IDs and Twilio values here
```



```

INSTANCE_ID = ""
QUARANTINE_SG = ""

TWILIO_ACCOUNT_SID = ""
TWILIO_AUTH_TOKEN = ""
FROM_NUMBER = ""
TO_NUMBER = ""
# =====

try:
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'],
encoding='utf-8')

    response = s3_client.get_object(Bucket=bucket_name, Key=file_key)
    compressed_body = response['Body'].read()

    if file_key.endswith('.gz'):
        print("Detected Gzip compression. Extracting archive contents...")
        with gzip.GzipFile(fileobj=io.BytesIO(compressed_body)) as gzip_file:
            raw_log = gzip_file.read().decode('utf-8')
    else:
        raw_log = compressed_body.decode('utf-8')

    # Split the file contents into individual log lines
    log_lines = raw_log.splitlines()
    print(f"Processing a batch of {len(log_lines)} logs from WAF storage...")

except Exception as parse_error:
    print(f"Failed to fetch or parse log object from S3: {str(parse_error)}")
    return

# Loop through EVERY entry inside the log file to search for a block
threat_detected = False
attacker_ip = "Unknown IP Address"

for line in log_lines:
    if not line.strip():
        continue
    try:
        log_data = json.loads(line)
        if log_data.get('action') == 'BLOCK':
            threat_detected = True
            http_request = log_data.get('httpRequest', {})

```

```

        attacker_ip = http_request.get('clientIp', "Unknown IP Address")
        break # Threat found! Stop searching the file and isolate immediately
except Exception as e:
    print(f"Error reading line: {str(e)}")
    continue

if not threat_detected:
    print("Log batch verified clean. No blocked malicious entries found in this payload.")
    return

# Proceed with isolation if a BLOCK was found
instance_name = "Production Infrastructure"
try:
    instance = ec2_resource.Instance(INSTANCE_ID)
    for tag in instance.tags or []:
        if tag['Key'] == 'Name':
            instance_name = tag['Value']
            break
except Exception as tag_err:
    print(f"Name Tag lookup failed: {str(tag_err)}")

print(f"🚨 WAF Block Extracted from batch! Target: {instance_name} | Attacker: {attacker_ip}")

try:
    ec2_client.modify_instance_attribute(
        InstanceId=INSTANCE_ID,
        Groups=[QUARANTINE_SG]
    )
    print(f"Network isolated cleanly for {instance_name}.")

    trigger_voice_alert(instance_name, attacker_ip, TWILIO_ACCOUNT_SID,
        TWILIO_AUTH_TOKEN, FROM_NUMBER, TO_NUMBER)

except Exception as e:
    print(f"Containment execution wrapper failed: {str(e)}")

def trigger_voice_alert(instance_name, attacker_ip, sid, token, from_num, to_num):
    clean_name = str(instance_name).replace("<", "").replace(">", "").replace("&",
    "and").replace("-", " dash ").replace("_", " underscore ")
    clean_ip = str(attacker_ip).replace(".", " dot ")

    speech_text = (
        f"Security Alert. The inline web application firewall has blocked an attack "

```

```

        f"against your {clean_name} infrastructure. The threat originated from I P address
        {clean_ip}. "
        f"The containment engine has successfully isolated the target server."
    )

    twiml_payload = f"<Response><Say voice=\"alice\">{speech_text}</Say></Response>"
    payload_dict = {"To": to_num, "From": from_num, "Twiml": twiml_payload}
    post_data = urllib.parse.urlencode(payload_dict).encode('utf-8')

    twilio_url = f"https://api.twilio.com/2010-04-01/Accounts/{sid}/Calls.json"
    auth_str = f"{sid}:{token}"
    b64_auth = base64.b64encode(auth_str.encode()).decode()

    req = urllib.request.Request(
        twilio_url,
        data=post_data,
        headers={
            'Authorization': 'Basic ' + b64_auth,
            'Content-Type': 'application/x-www-form-urlencoded'
        }
    )

    with urllib.request.urlopen(req) as response:
        print("Twilio call gateway accepted structured request payload: ",
        response.read().decode())

```

6. **CRITICAL:** Fill in your configuration credentials block variables accurately, and click the orange **Deploy** button.

3. Connect the S3 Lambda Event Trigger

1. Search for **S3** in the top search bar, click on your aws-waf-logs-soar-production bucket, and enter the **Properties** tab.
2. Scroll down to **Event notifications** click **Create event notification**.
3. **Event name:** Type `soar_s3_to_lambda_universal`.
4. Leave *Prefix* and *Suffix* completely, entirely **blank**.
5. Under **Event types**, check the box for **All object create events** (`s3:ObjectCreated:*`).

6. Scroll down to **Destination** choose **Lambda Function**, pick your `soar_production_containment_engine`.
7. Click **Save changes** at the bottom.